

Software Requirements Specification (SRS)

Full SMS Web Application

Version: 1.0

Date: May 22, 2026

Team: CoreTech

Client: University of Pretoria Biophysics Group

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) document describes the functional and non-functional requirements for the web-based version of Full_SMS (Single-Molecule Spectroscopy Analysis Software). The document is intended for the development team, project stakeholders, and the client to establish a shared understanding of the system's functionality, constraints, and expected behavior.

1.2 Project Scope

Full_SMS Web is a browser-based scientific data analysis platform designed to improve accessibility to single-molecule spectroscopy (SMS) analysis tools. The system enables biophysics researchers to upload, analyze, visualize, and share fluorescence measurement data from any location without requiring local software installation or high-performance workstations.

The existing desktop version of Full_SMS, developed by the University of Pretoria Biophysics Group, is a DearPyGui-based application that performs advanced analysis tasks including:

- Change point detection using the Watkins and Yang algorithm
- Hierarchical clustering using AHCA with BIC optimization
- Fluorescence lifetime fitting through multi-exponential decay analysis
- Auto-correlation and cross-correlation $g(2)$ analysis
- Spectra and raster scan visualization

Current Challenges

The desktop application currently presents several limitations:

- Installation complexity due to Python environment configuration and dependency management
- Restricted access to machines where the software is installed
- Limited collaboration capabilities between researchers
- Inability to remotely monitor long-running analyses
- Computational limitations based on local workstation hardware

Proposed Solution

The web-based platform addresses these limitations through:

- Browser-based access from any device
- No local installation requirements
- Shared datasets and collaborative analysis sessions
- Persistent sessions that can be resumed later
- In-browser and email notifications for analysis completion

1.3 Project Objectives

The primary objectives of the Full_SMS Web application are:

1. Improve accessibility by enabling SMS analysis through a web browser without software installation
 2. Support collaboration through secure dataset sharing and shared analysis workflows
 3. Provide an intuitive user interface with a minimal learning curve
 4. Improve reproducibility through detailed reports containing parameters and analysis results
 5. Support scalability for concurrent users and computationally intensive analysis tasks
 6. Ensure compatibility with HDF5 files while allowing future extensibility for additional formats
-

2. User Stories and User Characteristics

2.1 User Roles

The system is designed for two primary user groups.

2.1.1 Researcher (Primary User)

Researchers are academic or industry scientists conducting single-molecule spectroscopy experiments. They generate HDF5 measurement files from laboratory equipment and require tools for advanced analysis, including change point detection, clustering, lifetime fitting, and correlation analysis.

Technical Proficiency

- Moderate to advanced scientific computing knowledge
- Familiar with data analysis concepts
- Limited software development experience may be expected

Usage Characteristics

- Regular usage ranging from daily to weekly
- Ability to run multiple analyses concurrently
- Requirement for publication-quality exports
- Need for reproducible workflows and reporting

2.1.2 Collaborator (Secondary User)

Collaborators are researchers who are granted access to shared datasets and analysis sessions by other users. Their permissions may range from read-only access to limited editing capabilities.

Technical Proficiency

- Similar scientific background to primary researchers
- May be less familiar with the originating experimental setup

Usage Characteristics

- Occasional access for collaborative projects
- Primarily focused on viewing and reviewing analyses
- May export results for presentations or publications
- Restricted access to explicitly shared datasets

2.2 User Stories

The following user stories describe the system requirements from the perspective of end users.

2.2.1 Authentication and Access Control

US-01

As a researcher, I want to be able to create an account and securely log into the platform so that my research data remains private and protected.

US-02

As a researcher, I want to invite collaborators to access my datasets so that my team members can view and review my analysis work.

US-03

As a collaborator, I want to load previously saved sessions so that I can review shared analysis work.

US-04

As a collaborator, I want to request edit permissions for shared datasets so that I can contribute to collaborative analyses.

2.2.2 File Management and Data Upload

US-05

As a researcher, I want to upload HDF5 measurement files so that I can begin analysis without transferring files to another workstation.

US-06

As a researcher, I want support for multiple file formats so that I can work with different experimental datasets.

US-07

As a researcher, I want to share datasets with collaborators without requiring additional software installation.

2.2.3 Session Management

US-08

As a researcher, I want to save my analysis session so that I can continue my work later without repeating completed steps.

US-09

As a researcher, I want to access the platform through any browser so that I can monitor analyses remotely.

2.2.4 Analysis and Processing

US-10

As a researcher, I want analysis tasks to run in the background so that I can continue working on other datasets simultaneously.

US-11

As a researcher, I want to configure default analysis parameters so that repetitive tasks require less manual setup.

US-12

As a researcher, I want to receive notifications when long-running analyses are complete so that I can manage my workflow efficiently.

US-13

As a researcher, I want to navigate between distinct tabs for Intensity, Lifetime, Grouping, Correlation, Spectra, Raster, and Export so that I can execute the complete standardized SMS analysis workflow.

US-14

As a researcher, I want to view and interact with zoomable plots within each analysis view so that I can inspect high-precision spectroscopy data details.

2.2.5 Data Export and Reporting

US-15

As a researcher, I want to export plots as high-resolution PNG images so that I can include them in presentations and publications.

US-16

As a researcher, I want to export multiple files in a single operation so that I can save time when preparing analysis outputs.

US-17

As a researcher, I want to generate a formatted PDF report containing analysis parameters and graphs so that I have a reproducible record of the session.

US-18

As a researcher, I want to export my processed analysis results into CSV and HDF5 formats so that I can utilize them in external tools or retain native data structures.

2.2.6 Usability and User Experience

US-19

As a researcher, I want the platform to be easy to learn and use so that I can focus on scientific analysis rather than software operation.

2.2.7 Data Integrity and Accountability

US-20

As a researcher, I want the system to maintain a detailed log of all analysis actions performed on each dataset to guarantee reproducibility and workflow accountability.

2.3 User Priorities

The following priorities have been identified for the initial implementation phase.

High Priority

- Authentication and secure login
- HDF5 file upload support
- Session save and load functionality
- Background analysis processing
- Basic plot export functionality

Medium Priority

- Support for additional file formats
- Dataset sharing and collaboration
- Batch export functionality
- PDF report generation

Lower Priority

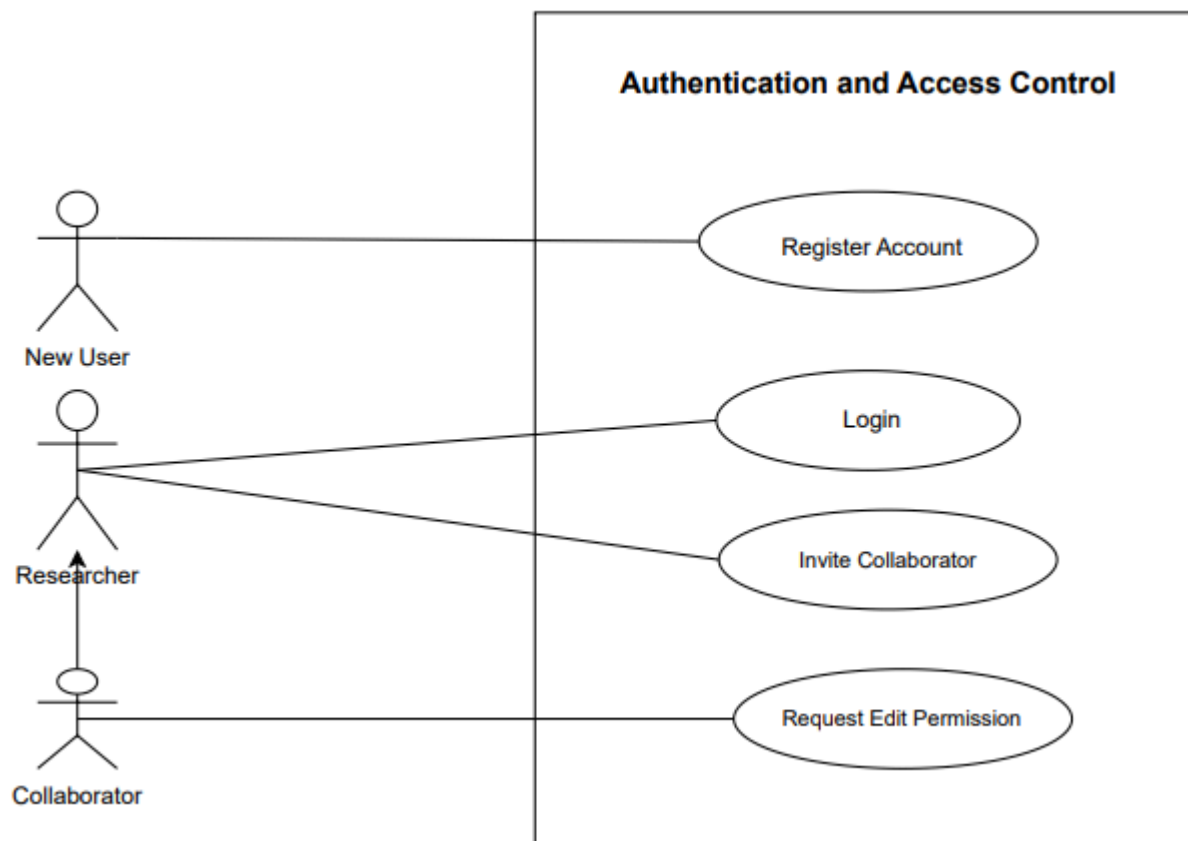
- Default parameter configuration
- Advanced collaboration features
- Email and browser notifications

3. Use Cases

This section describes the primary use cases derived from the user stories and functional requirements. Each use case defines the event that initiates the interaction and the expected outcome once the interaction is completed.

3.1 Use Case Descriptions

3.1.1 Authentication and Access Control



- Use Case: Register Account

Related User Stories: US-01

The Use Case Begins When:

A new user navigates to the Full_SMS web application and selects the “Register” option on the landing page.

The Use Case Ends When:

The system successfully creates the user account and redirects the user to the login page.

- Use Case: Login to System

Related User Stories: US-01, US-03

The Use Case Begins When:

A registered user enters their login credentials and selects the “Login” button.

The Use Case Ends When:

The user is authenticated successfully and gains access to the dashboard containing experiments that they are working on, this includes experiments they own and the ones they were invited to.

- **Use Case: Invite Collaborator**

Related User Stories: US-02, US-07

The Use Case Begins When:

A researcher selects the “Share” option for a dataset and enters a collaborator’s email address.

The Use Case Ends When:

The collaborator receives an invitation to access the dataset, and the researcher receives confirmation that the invitation was sent successfully.

- **Use Case: Request Edit Permission**

Related User Stories: US-04

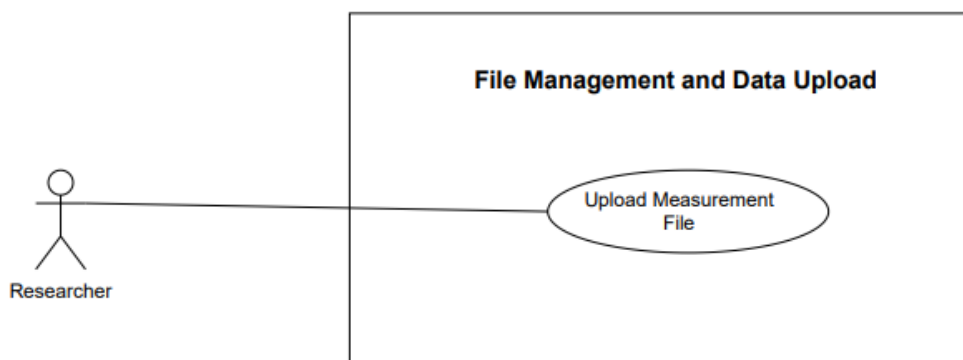
The Use Case Begins When:

A collaborator viewing a shared dataset selects the “Request Edit Access” option.

The Use Case Ends When:

The dataset owner receives a notification requesting edit permissions for the collaborator.

3.1.2 File Management and Data Upload



- **Use Case: Upload Measurement File**

Related User Stories: US-05, US-06

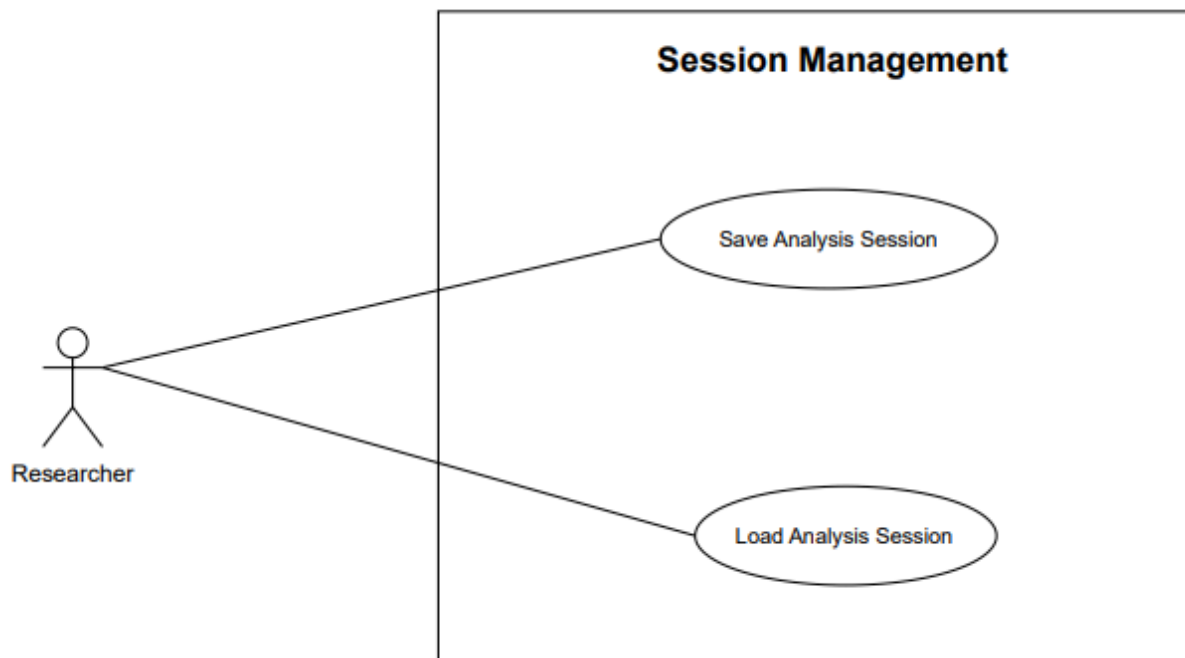
The Use Case Begins When:

An authenticated researcher selects the “Upload File” option and chooses an HDF5 measurement file from their local machine.

The Use Case Ends When:

The system uploads the file successfully, parses it according to the internal scientific data models, and displays measurement metadata, including photon counts, channel information, and acquisition parameters.

3.1.3 Session Management



- **Use Case: Save Analysis Session**

Related User Stories: US-08

The Use Case Begins When:

A researcher selects the “Save Session” option after completing one or more analyses.

The Use Case Ends When:

The current analysis session is stored successfully and appears in the user’s recent sessions list.

- **Use Case: Load Analysis Session**

Related User Stories: US-03, US-08, US-09

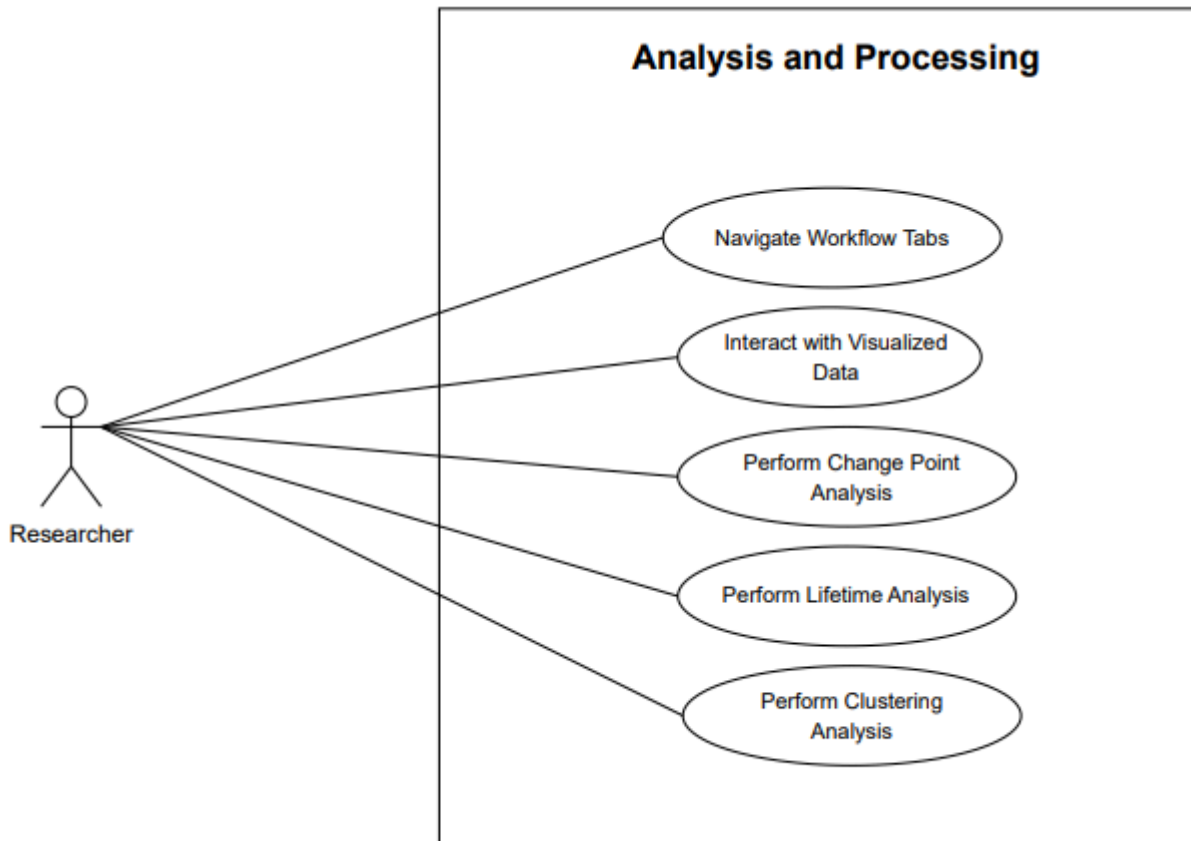
The Use Case Begins When:

A researcher or collaborator selects a saved session from the dashboard.

The Use Case Ends When:

The system restores the saved session, including datasets, computed results, analysis parameters, and visualizations.

3.1.4 Analysis and Processing



- **Use Case: Navigate Workflow Tabs**

Related User Stories: US-13

The Use Case Begins When: A researcher selects an active dataset and clicks on any of the core analysis pipeline tabs (*Intensity, Lifetime, Grouping, Correlation, Spectra, Raster, or Export*).

The Use Case Ends When: The application dynamically displays the selected scientific workflow view alongside its corresponding, context-specific plotting areas and parameter tools.

- **Use Case: Interact with Visualized Data**

Related User Stories: US-14

The Use Case Begins When: A researcher uses mouse gestures to zoom into, pan across, or select specific regions on any generated spectroscopy plot.

The Use Case Ends When: The plot updates in high resolution instantly to isolate the selected data window, allowing high-precision inspection of data details.

- **Use Case: Perform Change Point Analysis**

Related User Stories: US-10, US-11, US-12, US-13

The Use Case Begins When: A researcher navigates to the Intensity view, configures processing parameters, and chooses the “Run Change Point Analysis” option.

The Use Case Ends When: The system runs the algorithm asynchronously in the background, delivers a completion notification, and displays the results as an interactive intensity plot with identified photon brightness levels.

- **Use Case: Perform Lifetime Analysis**

Related User Stories: US-10, US-11, US-12, US-13

The Use Case Begins When: A researcher navigates to the Lifetime view and selects the “Fit Lifetime Decay” option after defining the active lifetime constraints.

The Use Case Ends When: The system processes the calculation asynchronously and displays the resulting fluorescence lifetime decay histogram, fitted decay curves, and calculated fitting parameters.

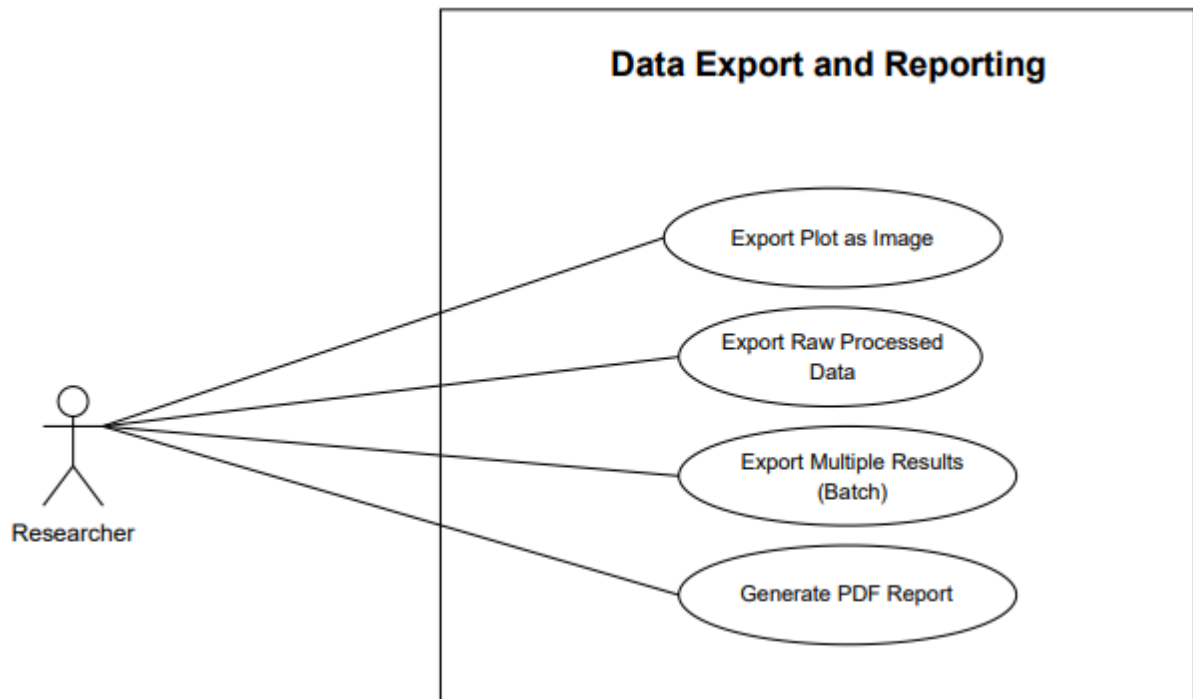
- **Use Case: Perform Clustering Analysis**

Related User Stories: US-10, US-11, US-12, US-13

The Use Case Begins When: A researcher navigates to the Grouping view and selects the “Cluster Levels” option to sort identified intensity steps.

The Use Case Ends When: The system displays the hierarchical clustering dendrogram and optimized cluster groupings calculated using Bayesian Information Criterion (BIC) analysis.

3.1.5 Data Export and Reporting



- **Use Case: Export Plot as Image**

Related User Stories: US-15

The Use Case Begins When: A researcher selects the “Export” option while viewing an individual analysis plot and chooses PNG as the export format.

The Use Case Ends When: The system generates and downloads a high-resolution PNG image containing the current plot view and associated labels suitable for publication.

- **Use Case: Export Raw Processed Data**

Related User Stories: US-18

The Use Case Begins When: A researcher selects the “Export” option under the dedicated Export tab and chooses CSV or HDF5 as the output data type.

The Use Case Ends When: The system compiles the processed matrixes and results into the chosen file format and downloads it directly to the local storage machine.

- **Use Case: Export Multiple Results (Batch)**

Related User Stories: US-16

The Use Case Begins When: A researcher marks multiple generated analysis assets or sessions from the Export interface and selects the “Batch Export” option.

The Use Case Ends When: The system compresses all selected exports together and downloads a structured ZIP archive containing the requested parameters and datasets.

- Use Case: Generate PDF Report

Related User Stories: US-17

The Use Case Begins When: A researcher selects the “Generate Report” option under the Export tab after finalizing an entire session workflow.

The Use Case Ends When: The system compiles all historical analysis configurations, input parameters, result summaries, and graphical data plots into a cleanly formatted PDF report document for academic distribution.

4. Functional Requirements

R1: User Management & Security

- **R1.1: User Authentication**
 - **R1.1.1:** The system shall allow users to create accounts using a valid email address and password.
 - **R1.1.2:** The system shall enforce password strength requirements (minimum length, uppercase, numeric, and special characters).
 - **R1.1.3:** The system shall validate email address format during registration.
 - **R1.1.4:** The system shall authenticate users using email and password credentials.
 - **R1.1.5:** The system shall issue and maintain authenticated user sessions after successful login.
 - **R1.1.6:** The system shall provide a secure password reset mechanism using time-limited reset links.
 - **R1.1.7:** The system shall invalidate password reset links after use or expiration.
- **R1.2: Role-Based Access Control (RBAC)**
 - **R1.2.1:** The system shall support at least two user roles: Administrator and Researcher.
 - **R1.2.2:** The system shall restrict protected functionality to authenticated users only.
 - **R1.2.3:** The system shall enforce role-based permissions for datasets, analysis tools, and administrative features.
 - **R1.2.4:** The system shall deny unauthorized access attempts.
 - **R1.2.5:** Administrators shall be able to assign, modify, and revoke user roles dynamically.
- **R1.3: Session Persistence**
 - **R1.3.1:** The system shall allow authenticated users to save analysis sessions.

- **R1.3.2:** Saved sessions shall include dataset references, analysis parameters, regions of interest, and results.
- **R1.3.3:** Users shall be able to resume saved sessions from any supported browser.
- **R1.3.4:** The system shall display a list of previously saved sessions (name, dataset, timestamp) for each user.
- **R1.4: Default Analysis Parameters**
 - **R1.4.1:** The system shall allow users to save reusable analysis parameter configurations (e.g., binning width, clustering bounds).
 - **R1.4.2:** Users shall be able to load saved configurations before analysis execution.
- **R1.5: Dataset Sharing**
 - **R1.5.1:** The system shall allow dataset owners to share datasets with other registered users.
 - **R1.5.2:** Dataset sharing shall support multiple access levels (read-only, edit).
 - **R1.5.3:** Dataset owners shall be able to revoke shared access at any time.

R2: Data Management & Ingestion

- **R2.1: File Upload**
 - **R2.1.1:** Authenticated users shall be able to upload spectroscopy data in .pt3 and .csv formats.
 - **R2.1.2:** The system shall reject unsupported file types and display upload progress.
 - **R2.1.3:** The system shall automatically intercept and reject unsupported file type extensions.
 - **R2.1.4:** The system shall track and display upload progress for each file added to the queue.
- **R2.2: Automatic Data Conversion**
 - **R2.2.1:** The system shall automatically convert uploaded datasets into internal HDF5 (.h5) format.
 - **R2.2.2:** The system shall notify users upon conversion completion or failure.
- **R2.3: Persistent Storage**
 - **R2.3.1:** The system shall store converted datasets and metadata (size, owner, timestamp) in centralized storage.
- **R2.4: Data Export**
 - **R2.4.1:** Users shall be able to export results in CSV, PNG, and Parquet formats.
- **R2.5: Bulk / Batch Export**
 - **R2.5.1:** The system shall support simultaneous multi-dataset exports packaged as ZIP archives.
- **R2.6: Temporary File Management**
 - **R2.6.1:** The system shall automatically purge temporary conversion files after a retention period.

R3: Signal Analysis & User Controls

- **R3.1: Region of Interest (ROI) Selection**
 - **R3.1.1:** Users shall define ROIs on photon traces using start and end time values.
- **R3.2: Artifact Filtering**
 - **R3.2.1:** The system shall provide automated artifact detection (spikes/bursts) and manual marking.
- **R3.3: Automated State Detection**
 - **R3.3.1:** The system shall perform Change-Point Analysis (CPA) and cluster detected states.
- **R3.4: Parameter Adjustment**
 - **R3.4.1:** Users shall adjust binning width, clustering bounds, and thresholds via input controls.
- **R3.5: Background Processing**
 - **R3.5.1:** Intensive analyses (clustering, fitting) shall execute asynchronously via task queues.
- **R3.6: Multi-Exponential Lifetime Fitting**
 - **R3.6.1:** The system shall support configurable multi-exponential fluorescence lifetime fitting.
- **R3.7: AHCA Clustering**
 - **R3.7.1:** The system shall support Agglomerative Hierarchical Clustering Analysis (AHCA).

R4: Visualisation & Reporting

- **R4.1: Interactive Time-Trace Display**
 - **R4.1.1:** The system shall render zoomable time-trace plots for single and dual-channel datasets.
- **R4.2: Raster Scan Image Rendering**
 - **R4.2.1:** The system shall generate 2D raster scans with interactive tooltips for coordinates and intensity.
- **R4.3: Statistical Chart Generation**
 - **R4.3.1:** The system shall generate intensity histograms, BIC plots, and decay curves.
- **R4.4: PDF Report Generation**
 - **R4.4.1:** Users shall export analysis sessions as downloadable PDF reports.
- **R4.5: Notification System**
 - **R4.5.1:** The system shall provide in-browser and optional email notifications for task status.

R5: Administrative & Monitoring Features

- **R5.1: Job / Queue Monitoring**
 - **R5.1.1:** Administrators shall have an interface to monitor and cancel active, pending, or completed jobs.

- **R5.2: Audit & Activity Logging**
 - **R5.2.1:** The system shall maintain immutable logs of all dataset actions (uploads, edits, sharing).

5. Quality (Non-Functional) Requirements

Quality requirements (non-functional requirements) define system characteristics and constraints that are not directly related to functional behavior. These requirements ensure the system is reliable, performant, secure, usable and maintainable.

NFR1: Performance Requirements

- **NFR1.1: Page Load Performance**
 - **NFR1.1.1:** The system shall load pages within acceptable timeframes on standard internet connections.
 - **NFR1.1.2:** The system shall respond to API requests with minimal latency for a majority of requests.
- **NFR1.2: File Upload Capability**
 - **NFR1.2.1:** The system shall support uploads of large files with real-time progress tracking.
- **NFR1.3: Concurrent User Capacity**
 - **NFR1.3.1:** The system shall maintain consistent performance across multiple concurrent user sessions without noticeable degradation.
- **NFR1.4: Background Processing Efficiency**
 - **NFR1.4.1:** The system shall complete background analysis tasks within a reasonable timeframe proportional to algorithm complexity.

NFR2: Reliability Requirements

- **NFR2.1: System Availability**
 - **NFR2.1.1:** The system shall maintain high availability and minimize unexpected downtime.
- **NFR2.2: Task Recovery**
 - **NFR2.2.1:** The system shall automatically recover from failed asynchronous tasks within acceptable timeframes.
- **NFR2.3: Data Backup & Retention**
 - **NFR2.3.1:** The system shall perform regular automated backups of all data.
 - **NFR2.3.2:** The system shall retain backup copies for an extended retention period to enable recovery.
- **NFR2.4: Session Persistence**
 - **NFR2.4.1:** The system shall preserve user session data across infrastructure restarts.

NFR3: Scalability Requirements

- **NFR3.1: Large Dataset Support**
 - **NFR3.1.1:** The system shall handle analysis of large-scale datasets.
- **NFR3.2: User Data Capacity**
 - **NFR3.2.1:** The system shall store and manage a substantial number of datasets per user without performance degradation.
- **NFR3.3: Horizontal Scaling**
 - **NFR3.3.1:** The system shall support horizontal scaling through containerization and load distribution.
- **NFR3.4: Storage Capacity**
 - **NFR3.4.1:** The system shall scale storage infrastructure to accommodate organizational growth.

NFR4: Security Requirements

- **NFR4.1: Data Encryption in Transit**
 - **NFR4.1.1:** The system shall encrypt all network communications using industry-standard encryption protocols.
- **NFR4.2: Authentication Token Management**
 - **NFR4.2.1:** The system shall issue authentication tokens with defined expiration periods.
 - **NFR4.2.2:** The system shall provide mechanisms for token refresh without re-authentication.
- **NFR4.3: Password Security**
 - **NFR4.3.1:** The system shall store passwords using industry-standard cryptographic hashing algorithms such as ES256
- **NFR4.4: SQL Injection Prevention**
 - **NFR4.4.1:** The system shall prevent SQL injection attacks through parameterized query execution.
- **NFR4.5: Cross-Site Scripting (XSS) Protection**
 - **NFR4.5.1:** The system shall implement security headers to prevent cross-site scripting attacks.
- **NFR4.6: Cross-Origin Resource Sharing (CORS)**
 - **NFR4.6.1:** The system shall restrict cross-origin requests to approved sources only.
- **NFR4.7: File Upload Validation**
 - **NFR4.7.1:** The system shall validate uploaded files against a whitelist of permitted file types.

NFR5: Maintainability Requirements

- **NFR5.1: Code Coverage**
 - **NFR5.1.1:** The system shall maintain adequate automated test coverage for backend and frontend components.
- **NFR5.2: API Documentation**
 - **NFR5.2.1:** The system shall provide comprehensive API documentation in a standardized format.
- **NFR5.3: Code Documentation**

- **NFR5.3.1:** The system shall include documentation for all functions and classes via inline comments.
- **NFR5.4: Continuous Integration & Deployment**
 - **NFR5.4.1:** The system shall validate builds automatically before deployment to production.
- **NFR5.5: Error Logging**
 - **NFR5.5.1:** The system shall capture detailed error information including stack traces and contextual data for debugging.

NFR6: Usability Requirements

- **NFR6.1: Browser Compatibility**
 - **NFR6.1.1:** The system shall function consistently across modern versions of major web browsers.
- **NFR6.2: User-Friendly Error Messages**
 - **NFR6.2.1:** The system shall display error messages using clear, non-technical language.
- **NFR6.3: Loading State Feedback**
 - **NFR6.3.1:** The system shall provide visual feedback for operations that require noticeable processing time.

6. API Service Contracts

This is a documentation of the system's RESTful API endpoints, request/response JSON schemas and error codes.

Base URL: <http://localhost:8000/api/py>

1. File Upload:

- Endpoint: POST/upload/
- Description: Upload a spectroscopy data file to the server.
- Authentication: Not required (will be added in the future sprint)
- Content-Type: multipart/form-data

Format of a request:

Field	Type	Required	Description
file	file	Yes	File to upload (.pt3, .csv, .h5, .hdf5)

Success Response "200 OK"

JSON code:

```
{ "message": "File uploaded successfully",
  "filename": "experiment.pt3",
  "saved_as": "a3f9c2d1_experiment.pt3",
  "size_bytes": 204800,
  "status": "pending" }
```

}

Error Responses:

Code	Reason
400	Unsupported file type
422	No file provided in request

Error Response Body “400”:

JSON code:

```
{ "detail": "Unsupported file type: '.pdf'. Allowed: {'.pt3', '.csv', '.h5', '.hdf5'}" }
```

2. Verify Authentication Token

- Endpoint: POST/auth/verify-token
- Description: Verify a Supabase JWT token sent from the frontend. Supports both ES256 and HS256 algorithms.
- Authentication: Required, bearer token in header
- Content-Type: application/json

Request Header:

Header	Value
Authorization	Bearer <JWT token>

Success Response “200 OK”

JSON code:

```
{ "valid": true,  
  "user_id": "abc123" }
```

Error Responses:

Code	Reason
401	Token has expired or is invalid
500	SUPABASE_JWT_SECRET or SUPABASE_JWK not configured

Error Response Body 401

JSON code:

```
{ "detail": "Token has expired" }
```

3. Get User Profile

- Endpoint: GET/profile/me
- Description: Retrieve the currently authenticated user’s profile from Supabase
- Authentication: Required, bearer token in authorization header
- Content-Type: application/json

Request Header:

Header	Value
Authorization	Bearer <JWT token>

Success Response “200 OK”**JSON code:**

```
{
  "id": "abc123",
  "email": "researcher@up.ac.za",
  "username": "john_doe",
  "role": "researcher"
}
```

Error Responses:

Code	Reason
401	Not authenticated
404	User not found in Supabase
500	Supabase environment variables not configured

Error Response Body 404:

```
{
  "Detail": "User not found"
}
```

4. Update User Profile

- Endpoint: PUT/profile/me
- Description: Update the currently authenticated user's username and/or password in Supabase
- Authentication: Required, bearer token in authorization header
- Content-Type: application/json

Request Header:

Header	Value
Authorization	Bearer <JWT token>

Request Body

Field	Type	Required	Description
username	string	No	New username

new_password	string	No	New password
--------------	--------	----	--------------

Example of a user updating their password:

Request sent will be:

```
{
  "username": "john_doe",
  "new_password": "newSecurePassword123"
}
```

And the response will be a **Success Response “200 OK”**:

```
{
  "id": "abc123",
  "email": "researcher@up.ac.za",
  "username": "john_doe",
  "role": "researcher"
}
```

Error Responses:

Code	Reason
400	No update data provided
401	Not authenticated
500	Failed to update user in Supabase

5. Read HDF5 File

- Endpoint: POST/hdf5/read
- Description: Upload and read an HDF5 spectroscopy file. Returns metadata and measurement summaries.
- Authentication: Required
- Content-Type: multipart/form-data

Request Body:

Field	Type	Required	Description
File	File	Yes	HDF5 file to read (.h5 or .hdf5)

Success Response “200 OK”:

JSON code:

```
{
  "metadata": {
    "filename": "experiment.h5",
    "num_measurements": 5,
    "has_spectra": true,

```

```

    "has_rasters": false
  },
  "measurements": [
    {
      "id": 1,
      "name": "Measurement 1",
      "channelWidth": 0.016,
      "description": "Single molecule experiment"
    }
  ]
}

```

Error Responses

Code	Reason
400	Invalid or unreadable HDF5 file
422	No file provided in request

6. Health Check

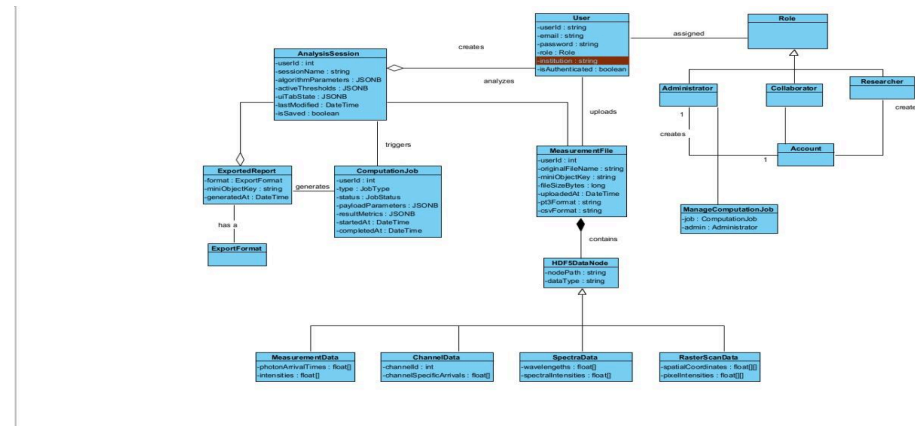
- Endpoint: GET/health
- Description: Verifies that the API is running correctly
- Authentication: Not required

Success Response “200 OK”

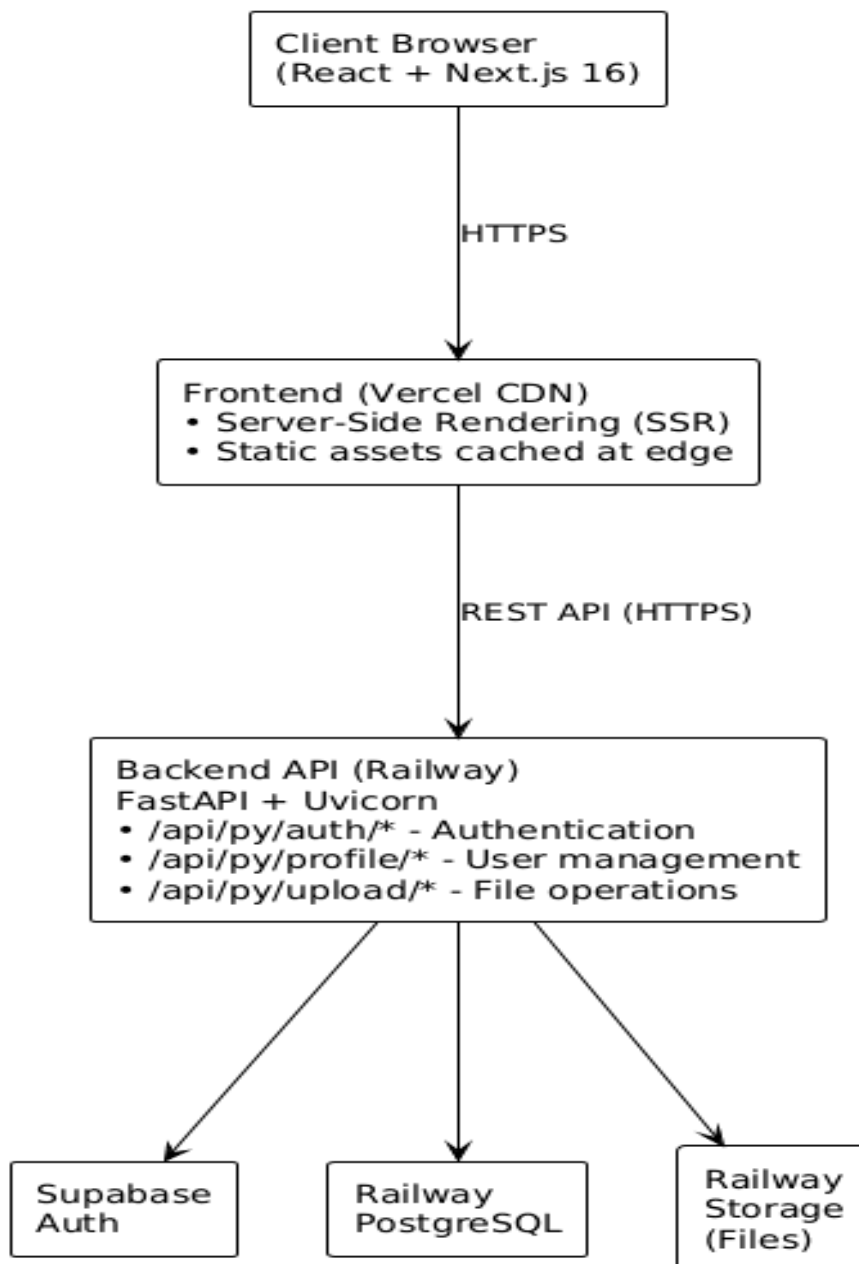
JSON code:

```
{
  "status": "ok",
  "message": "Full-SMS API is running!"
}
```

7. Domain Model



8. Preliminary Architecture Diagram



9. Technology Requirements

- **Back-end technologies used:**
 - **Supabase:** This is an open-source backend-as-a-service platform designed to help developers build and scale applications without the hassle of managing their own servers or complex backend infrastructure. It has a dedicated PostgreSQL database with an easy-to-use table editor that makes SQL management feel like using a spreadsheet. It also provides authentication features by having a built-in user management supporting email/password and social logins(Google, GitHub, etc.). Since we have not

yet implemented the database needed for this system, Supabase was mainly used for authentication.

- **Back-end technologies used:**

- **FastAPI:** This is a modern, high-performance web framework used for building RESTful APIs with Python. Researchers prefer using python for coding activities during their analysis simulations and other researching tasks. As a result, FastAPI was the best choice for building the backend of this modern web application.
- **Railway:** This is a full-stack cloud for deploying web applications, servers, databases and more with automatic scaling, monitoring and security. It allows the team to preview every pull request automatically. It also allows us to rollback to any previous version of the software instantly when something breaks.
- **Pytest:** This is an open-source testing frame for the Python programming language that is used to write and execute various types of software tests, including unit, integration and functional tests. The original desktop has 577 passing scientific tests, pytest allows us to run the identical test suite against our new API, guaranteeing 100% numerical equivalence between the desktop and the web version.

- **Front-end technologies used:**

- **Next.js:** This is a React framework that enables several extra features including server-side rendering. Having a server-side rendering feature enables a fast initial loading of analysis dashboards. This is very beneficial for researchers with large datasets and need immediate visual feedback without waiting for client-side JavaScript to parse everything.
- **Jest:** This is a JavaScript testing framework with a focus on simplicity. It's fast, easy to set up and works seamlessly with modern JavaScript frameworks such as React and Next.js. It requires minimal configuration, supports isolated testing, and provides powerful assertion tools called "matchers". It ensures that React components don't unexpectedly change, maintaining UI consistency across the tabs.
- **Vercel:** This is a cloud platform that allows developers to build, deploy, and scale web applications instantly. Deployments can be previewed for every pull request allowing the team to demonstrate new analysis features to the client before merging, ensuring client-approval before code reaches production.

10. Constraints

This section defines the boundaries within which the Full SMS Web Application must be designed and developed. They arise from academic, architectural, regulatory, and operational sources and cannot be changed without explicit approval from the client and project supervisors.

10.1 Technological Constraints

10.1.1 Frontend Technology

- The frontend must be implemented using [Next.js](#) (React framework) with TypeScript.
- Tailwind CSS must be used for styling.
- Jest and React Testing Library must be used for frontend testing, as configured in the repository.

10.1.2 Backend Technology

- The backend must be implemented in python using FASTAPI. This is required to allow direct reuse and integration of existing Full SMS scientific computation code.
- Authentication must be handled through Supabase. The team may not replace Supabase with a custom authentication system, as the Supabase JWT verification layer is already integrated into the backend service.
-

10.1.3 Database

- PostgreSQL must be used as the primary relational database. This is required for structured storage of user accounts, metadata, and season records.

10.1.4 Data Format

- The system must natively support HDF5 (.h5) as the primary scientific data format. This is a hard constraint imposed by the existing Full SMS analysis pipeline and the lab instrumentation that produces HDF5 measurement files.

10.2 Architectural Constraints

- The system must follow a strict frontend/backend separation. The existing desktop application tightly coupled with GUI and logic; the web-based replacement should not repeat this.
- The plugin system should use a standardised input template for all analysis algorithms. Existing Python analysis algorithms must be wrapped and exposed through this plugin interface rather than called directly.
- Long running computations must be executed asynchronously . Analysis tasks such as lifetime fitting and clustering must not block the main application thread or browser. A task queue mechanism must be used for background processing.